

C++ 프로그래밍

(로그 파일)

14주차

목차

- 로그 파일
- 설계
- 코딩

로그 파일

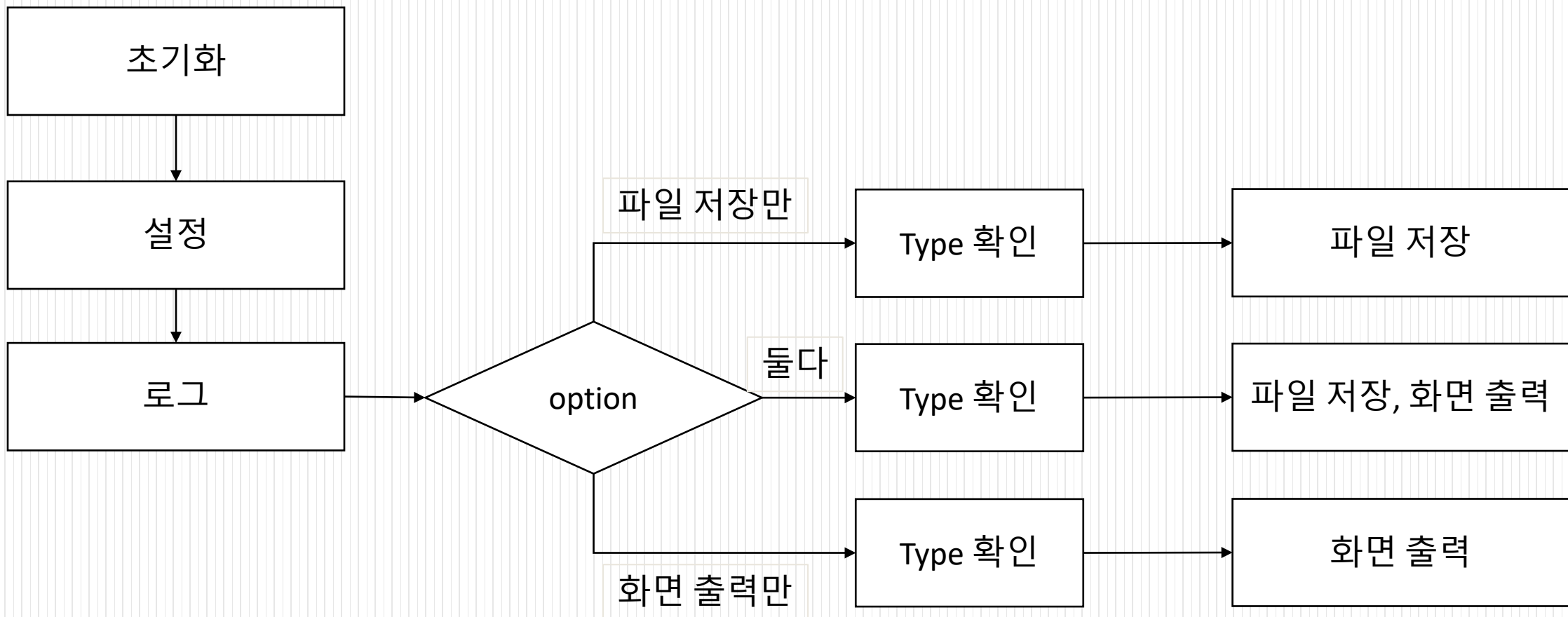
- 로그 파일의 정의
 - 운영 체제나 다른 소프트웨어가 실행 중에 발생하는 이벤트나 각기 다른 사용자의 통신 소프트웨어 간의 메시지를 기록하는 파일
- 로그의 종류
 - 이벤트 로그
 - 메시지 로그
 - 트랜잭션 로그
 - 에러 로그

로그 파일 (계속)

- 로그의 종류
 - 이벤트 로그
 - 시스템 활동을 이해하고 문제를 진단하는데 사용하는 로그
 - 트랜잭션 로그
 - 시스템과 해당 시스템 사용자 간 통신에 대한 파일이거나, 시스템 속하는 터미널의 사용자가 발생시킨 트랜잭션의 종류, 내용, 시간을 자동으로 캡처한 데이터 수집 방식
 - 에러 로그
 - 외부 문제로 인해 기능이 예상대로 작동하지 않는 경우 또는 잘못되었거나 비정상적인 프로그램 동작을 나타내는 프로그램 수준.

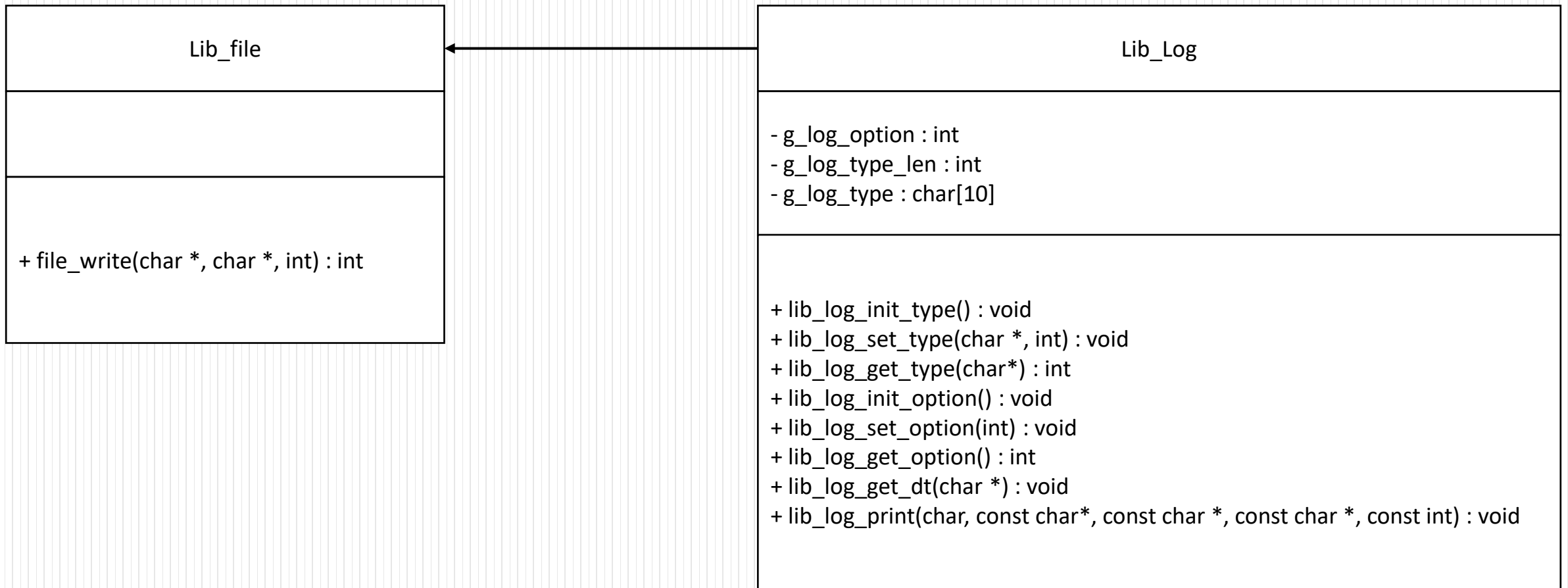
설계

- 전체 구성



설계 (설계)

- 클래스 다이어그램





- lib_file class

```
class lib_file {
public:
    lib_file()
    {
    }
    int file_write(char* in_path, char* in_data, int in_data_len);
};

int lib_file::file_write(char* in_path, char* in_data, int in_data_len)
{
    try
    {
        ofstream fd(in_path, ios_base::out | ios_base::app);

        fd.write(in_data, in_data_len);

        fd.close();

        return in_data_len;
    }
    catch (exception ex)
    {
        return -1;
    }
}
```

소스 (계속)

- lib_log class

```
class lib_log : public lib_file {  
  
private:  
    int g_log_option;  
    int g_log_type_len;  
    char g_log_type[10];  
  
public:  
    lib_log()  
    {  
        lib_log_init_type();  
        lib_log_init_option();  
    }  
    void lib_log_init_type();  
    void lib_log_set_type(char* in_type, int in_len);  
    int lib_log_get_type(char* out_type);  
    void lib_log_init_option();  
    void lib_log_set_option(int in_option);  
    int lib_log_get_option();  
    void lib_log_get_dt(char* out_dt);  
  
    void lib_log_print(char in_type, const char* in_log_data, const char* in_src_file, const char* in_func, const int in_line_no);  
  
};
```


소스 (계속)

- lib_log class (계속)

```
void lib_log::lib_log_init_type()
{
    memset(g_log_type, 0x00, sizeof(g_log_type));
    g_log_type_len = 0;
}

void lib_log::lib_log_set_type(char* in_type, int in_len)
{
    Memcpy(g_log_type, in_type, in_len);
    g_log_type_len = in_len;
}

int lib_log::lib_log_get_type(char* out_type)
{
    return g_log_type_len;
}

void lib_log::lib_log_init_option()
{
    g_log_option = 0;
}

void lib_log::lib_log_set_option(int in_option)
{
    g_log_option = in_option;
}
```

소스 (계속)

- lib_log class (계속)

```
int lib_log::lib_log_get_option()
{
    return g_log_option;
}

void lib_log::lib_log_get_dt(char* out_dt)
{
    char buf[128] = { 0, };
    char dt[300] = { 0, };
    struct timeb tb;
    struct tm tstruct;

    ftime(&tb);
    localtime_s(&tstruct, &tb.time);
    sprintf_s(dt, "%04d%02d%02d%02d%02d%d", tstruct.tm_year+1900, tstruct.tm_mon+1, tstruct.tm_mday,
    tstruct.tm_hour, tstruct.tm_min, tstruct.tm_sec, tb.millitm);

    memcpy(out_dt, dt, strlen(dt));
}
```

소스 (계속)

- lib_log class (계속)

```
void lib_log::lib_log_print(char in_type, const char* in_log_data, const char* in_src_file, const char* in_func, const int in_line_no)
{
    int cnt = 0, flag = 0, log_type_flag = 0, log_len = 0, log_type_len = 0;
    char log_type[30] = { 0, };
    char log_data[2048] = { 0, };
    char dt[30] = { 0, };
    char file_name[300] = { 0, };

    //time 구하기
    lib_log_get_dt(dt);
    //옵션 확인
    flag = lib_log_get_option();

    //type 확인
    log_type_len = lib_log_get_type(log_type);
    for (cnt = 0; cnt < log_type_len; cnt++)
    {
        if (log_type[cnt] == in_type)
        {
            log_type_flag = 0;
            break;
        }
        else
        {
            log_type_flag = 1;
        }
    }
}
```

소스 (계속)

- lib_log class (계속)

```
if (log_type_flag == 0)
{
    flag = 0;
}

sprintf_s(log_data, "[%c]:%s:FILENAME: %s, FUNCNAME: %s, LINENO: %d -> %sWrWn", in_type, dt, in_src_file, in_func, in_line_no, in_log_data);
log_len = strlen(log_data);

if ((flag == 1) || (flag == 3))
{
    printf("%s", log_data);
}

if ((flag == 2) || (flag == 3))
{
    file_write(file_name, log_data, log_len);
}
}
```

소스 (계속)

- main

```
#include <stdio.h>
#include <string.h>
#include <time.h>
#include <sys/timeb.h>
#include <iostream>
#include <fstream>
#include <sstream>

using namespace std;

int main()
{
    char log_type[30] = { 0, };

    lib_log *log = new lib_log();

    log->lib_log_set_option(1);
    log_type[0] = 'E' ;
    log->lib_log_set_type(log_type, 1);

    log->lib_log_print('E', "test", __FILE__, __FUNCTION__, __LINE__);
}
```